Dr. D. Y. Patil Technical Campus (DYPTC)

Talegaon-Varale, Pune • Department of Computer Engineering • DTE Code: 6991 (SPPU Affiliated)

DSA Lab Course Project
Academic Year 2026-27 | SEM-03

Smart Campus Lost & Found System

Complete Technical & Algorithmic Documentation (C++17 Core Engine)

A high-performance algorithmic matching and campus custody tracking platform built strictly using modern C++17, the Crow multi-threaded microframework, SQLite3 database persistence, and pure HTML5/CSS3 zero-JavaScript architecture.

COURSE NAME	COURSE CODE	CORE LANGUAGE	WEB ARCHITECTURE
Data Structures & Algorithms	**CS-210257 (SPPU)**	**ISO C++17 (Native)**	**Crow C++ Microframework**

1 प्रकल्प विहंगावलोकन (Project Overview & Problem Statement)

Dr. D. Y. Patil Technical Campus (DYPTC) च्या विस्तीर्ण संकुलामध्ये दररोज शेकडो विद्यार्थ्यांची मौल्यवान उपकरणे जसे की Laptops, AirPods, Scientific Calculators, Smart ID Cards, Hostel Keys आणि Backpacks हरवतात. पारंपारिक मॅन्युअल नोंदवहीत नोंद केल्याने वस्तू शोधण्यात अनेक दिवस वाया जायचे आणि बनावट दावे (fraudulent claims) होण्याचा धोका असायचा.

हा प्रश्न सोडवण्यासाठी आपण **C++17** मध्ये संपूर्ण **Data Structures & Algorithms (DSA) Engine** विकसित केला आहे. या प्रणालीमध्ये कोणतेही बाह्य थर्ड-पार्टी अल्गोरिदम्स किंवा Python/Node.js लायब्ररी न वापरता सर्व सॉर्टिंग, सर्चिंग, स्ट्रिंग साम्य (Levenshtein & Jaccard), आणि प्रायॉरिटी क्यू रँकिंग **शुद्ध C++ कोड** द्वारे चालवले जाते.

2 सर्व Sorting Algorithms चे सविस्तर विश्लेषण (Sorting Deep-Dive)

प्रकल्पामध्ये **३ स्वतंत्र Sorting Algorithms** प्रत्यक्ष विकसित केले आहेत. वापरकर्ते आणि शिक्षक `items.html` पेजवर रनटाईम मायक्रोसेकंद (µs), तुलनांची संख्या (Comparisons), आणि स्वॅप्स थेट तपासून पाहू शकतात:

२.१ Merge Sort (Divide & Conquer) • Production Default

`O(N log N) Stable`

कार्यपद्धती: Merge Sort हा Divide-and-Conquer तत्त्वावर चालतो. तो वेक्टरचे दोन समान अर्धे भाग करतो (`mid = (low + high) / 2`), दोन्ही भाग रिकर्सिव्हली सॉर्ट करतो, आणि शेवटी दोन पॉइंटर्स वापरून सॉर्टेड क्रमाने एकत्र (merge) करतो.

> **का वापरले? (Why Used?):**
> १. **गॅरंटीड परफॉर्मन्स:** डेटा कितीही मोठा असला तरी Best, Average आणि Worst सर्व परिस्थितींमध्ये Merge Sort चा वेळ $O(N \log N)$ च राहतो.
> २. **Stable Sorting:** दोन वस्तूंची तारीख किंवा नाव समान असल्यास, आधी नोंदवलेली वस्तू आधीच राहते (समान घटकांची अंतर्गत जागा बदलत नाही).
> ३. **Binary Search ची पूर्वतयारी:** Binary Search चालवण्यासाठी डेटा आधी सॉर्टेड असणे बंधनकारक आहे, ते काम Merge Sort करते.

कोड संदर्भ: `src/dsa_engine.hpp` (ओळ 295 ते 350) • `dsa::mergeSort()` , `dsa::merge()`

२.२ Insertion Sort (Adaptive Shifting) • Optimal for Small/Near-Sorted Data

`O(N) Best / O(N²) Worst`

कार्यपद्धती: पत्ते खेळताना आपण हातातील पाने सॉर्ट करतो, त्याप्रमाणे हा डावीकडील सॉर्टेड सब-अरेमध्ये नवीन घटकाला उजवीकडून डावीकडे शिफ्ट करत योग्य जागी बसवतो.

> **का वापरले? (Why Used?):**
> १. **जवळपास सॉर्टेड डेटामध्ये सर्वात वेगवान:** कॅम्पसमध्ये आधी १०० वस्तू सॉर्ट केलेल्या असताना नव्याने केवळ १-२ वस्तूंचीच नोंद होते. अशा वेळी संपूर्ण अरे पुन्हा सॉर्ट करण्यापेक्षा Insertion Sort फक्त नवीन घटकाला त्याच्या जागी सरकवतो, जे अवघ्या **$O(N)$ वेळेत** पूर्ण होते.
> २. **Zero Extra Memory ($O(1)$ Space):** Merge Sort प्रमाणे याला कोणताही तात्पुरता (auxiliary) वेक्टर लागत नाही.

कोड संदर्भ: `src/dsa_engine.hpp` (ओळ 246 ते 290) • `dsa::insertionSort()`

२.३ Bubble Sort (Adjacent Comparison) • Educational Benchmarking

`O(N) Best / O(N²) Worst`

कार्यपद्धती: अरेमधील लगतच्या (adjacent) दोन घटकांची तुलना करतो आणि क्रम चुकीचा असल्यास त्यांची अदलाबदल (swap) करतो. प्रत्येक फेरीत सर्वात मोठा घटक पाण्याच्या बुडबुड्याप्रमाणे उजव्या बाजूला स्थिरावतो.

> **का वापरले? (Why Used?):**
> १. **Early Exit Optimization:** आपण कोडमध्ये `swapped` फ्लॅग वापरला आहे. जर एका संपूर्ण फेरीत एकही स्वॅप झाला नाही, तर डेटा सॉर्ट झाला आहे हे ओळखून अल्गोरिदम लगेच थांबतो, ज्यामुळे अनावश्यक लूप्स टळतात ($O(N)$ Best Case).
> २. **लॅबमधील तुलनात्मक निकष:** $O(N^2)$ अल्गोरिदम मोठ्या डेटामध्ये किती जास्त वेळ आणि स्वॅप्स घेतो हे Merge Sort च्या $O(N \log N)$ सोबत तुलना करून मॅडमना दाखवण्यासाठी.

कोड संदर्भ: `src/dsa_engine.hpp` (ओळ 209 ते 238) • `dsa::bubbleSort()`

📊 Sorting Algorithms सर्वसमावेशक तुलनात्मक तक्ता (Master Comparison Table)

ALGORITHM	BEST CASE	AVERAGE CASE	WORST CASE	SPACE COMPLEXITY	STABLE?	प्रकल्पामधील नेमका उद्देश
Merge Sort	`O(N log N)`	`O(N log N)`	`O(N log N)`	`O(N)`	होय (Yes)	कॅटलॉगचा मुख्य सॉर्ट + Binary Search साठी पूर्व-सॉर्टिंग
Insertion Sort	`O(N)`	`O(N²)`	`O(N²)`	`O(1)` In-place	होय (Yes)	नवीन १-२ आयटम्स जोडल्यावर जलद अडॉप्टिव्ह सॉर्टिंग
Bubble Sort	`O(N)`	`O(N²)`	`O(N²)`	`O(1)` In-place	होय (Yes)	Lab Evaluation मध्ये Comparision आणि Swaps चे रनटाईम प्रात्यक्षिक

3 Searching Algorithms: Linear Search vs Binary Search

शोध कार्यासाठी सिस्टीममध्ये दोन तंत्रे दिली आहेत, जी `search.html` वर ॲडमिनला मायक्रोसेकंद अचूकतेने तुलना करून दाखवतात:

३.१ Linear Search (Multi-Field Sequential Scan)

O(N) Complexity

कार्यपद्धती: अरेमधील पहिल्या घटकापासून शेवटच्या घटकापर्यंत क्रमाने जात प्रत्येक आयटमची तुलना करतो.

> **का वापरले?:** विद्यार्थी जेव्हा वस्तू शोधतात, तेव्हा ते फक्त नाव नाही तर *Category, Location, Colour, आणि Date* असे अनेक निकष एकत्र निवडतात. अशा बहुआयामी अनसॉर्टेड डेटावर Linear Search अत्यंत लवचिकपणे काम करतो.

कोड संदर्भ: `src/dsa_engine.hpp` (ओळ 365 ते 415) • `dsa::linearSearch()`

३.२ Binary Search (Logarithmic Divide & Conquer)

O(log N) Complexity

कार्यपद्धती: डेटा आधी Merge Sort ने सॉर्ट केलेला असतो. अल्गोरिदम मध्य घटक (`mid`) तपासून डाव्या किंवा उजव्या अर्ध्या भागात जातो, ज्यामुळे प्रत्येक पायरीवर शोधक्षेत्र ५०% कमी होते.

> **का वापरले?:** जर १०२४ वस्तू नोंदवलेल्या असतील, तर Linear Search ला १०२४ वेळा तपासणी करावी लागेल, परंतु Binary Search **केवळ १० फेऱ्यांमध्ये ($2^{10} = 1024$)** अचूक वस्तू शोधून काढतो!

कोड संदर्भ: `src/dsa_engine.hpp` (ओळ 420 ते 480) • `dsa::binarySearchByName()`

4 Data Structures (डेटा स्ट्रक्चर्स): प्रकार, निवड व कारणे

४.१ Priority Queue (Max-Heap) • Top-K Auto Match Ranking

Push: O(log K) • Top: O(1)

C++ STL: `std::priority_queue<MatchResult, std::vector<MatchResult>, MatchComparator>`

> **का वापरले? (महत्त्वाचा Viva प्रश्न):**
> जर १०० हरवलेल्या आणि १०० सापडलेल्या वस्तू असतील, तर एकूण $100 imes 100 = 10,000$ जोड्या तयार होतात. संपूर्ण १०,००० जोड्या Quicksort ने सॉर्ट केल्यास $O(K \log K)$ वेळ लागेल. परंतु वापरकर्त्याला **फक्त Top 5 सर्वोत्तम जुळणाऱ्या जोड्याच** हव्या असतात. Max-Heap मुळे सर्वाधिक स्कोअर असलेला घटक थेट मुळाशी (Root) $O(1)$ वेळेत मिळतो आणि काढणे $O(\log K)$ वेळेत होते!

कोड संदर्भ: `src/dsa_engine.hpp` (ओळ 510 ते 580) • `dsa::findRankedMatches()`

४.२ Hash Table Indexing (`std::unordered_map` & `std::unordered_set`)

O(1) Average Lookup

१. Category Inverted Index (`dsa::buildCategoryIndex`): कॅटेगरीनुसार (उदा. `"Electronics" → [1, 4, 9]`) आयडींचे हॅश टेबल बनवले जाते, ज्यामुळे इलेक्ट्रॉनिक्स सिलेक्ट केल्यास पूर्ण डेटा न तपासता $O(1)$ मध्ये यादी मिळते.
२. Bilingual Engine (`html::tr`): इंग्रजी-मराठी शब्दकोश हॅश टेबलमध्ये साठवून रिअल-टाईम भाषांतर $O(1)$ वेळेत केले जाते.

कोड संदर्भ: `src/dsa_engine.hpp` (ओळ 150 ते 162), `src/html_renderer.hpp` (ओळ 173)

४.३ Dynamic Array (`std::vector`) • Contiguous RAM Cache

O(1) Random Access

कॅम्पसमध्ये वस्तूंची संख्या मर्यादित नसल्याने Static Array ऐवजी Vector वापरला आहे. Contiguous Memory Allocation मुळे हा CPU Cache-friendly आहे आणि Random Access $O(1)$ वेळेत देतो.

कोड संदर्भ: `src/models.hpp` (ओळ 15 ते 55) • `struct Item`

5 प्रगत स्ट्रिंग व मॅचिंग अल्गोरिदम्स (Specialized Algorithms)

५.१ Dynamic Programming: Levenshtein Distance ($O(M imes N)$)

विद्यार्थी वस्तूंची नावे टाकताना स्पेलिंगच्या चुका करतात (उदा. **"AirPods"** vs **"Earbuds"** किंवा **"Airpod"**). साध्या `==` ने हे मॅच होत नाही. हा DP अल्गोरिदम $M imes N$ आकाराचा मॅट्रिक्स तयार करून दोन शब्दांमधील Minimum Edit Operations (Insert, Delete, Substitute) मोजतो आणि Normalized Ratio (0.0 ते 1.0) मध्ये रूपांतरित करतो:

```
DP[i][j] = min({ DP[i-1][j] + 1,        // deletion
                 DP[i][j-1] + 1,         // insertion
                 DP[i-1][j-1] + cost });// substitution
```

५.२ Set Theory: Jaccard Token Similarity

जेव्हा शब्दांचा क्रम बदलतो (उदा. **"HP Silver Laptop"** आणि **"Laptop Silver HP"**), तेव्हा Levenshtein कमी गुण देतो. म्हणून आपण Jaccard Similarity वापरतो, जी शब्दांचे संच (Token Sets) करून मोजते:

$$\text{Jaccard Ratio} = |A \cap B| \div |A \cup B|$$

आपले Hybrid Matcher **६०% Jaccard आणि ४०% Levenshtein** एकत्र करून अचूक गुण काढते.

५.३ Weighted 6-Factor Multi-Attribute Scoring Engine

घटक (ATTRIBUTE)	WEIGHTAGE	अल्गोरिदम	कॅम्पस उद्देश
1. Category	**25%**	Exact Hash Match	इलेक्ट्रॉनिक्स वस्तू बॅगेसोबत चुकीने जुळू नये
2. Item Name	**25%**	Hybrid (60% Jaccard + 40% Levenshtein)	ब्रँड व उपकरणाचे नाव अचूक जुळवणे
3. Location	**15%**	Token Intersection + Substring	Central Library, Canteen, AI&DS Lab जुळवणे
4. Colour	**15%**	Levenshtein Edit Tolerance	रंगातील साम्य (Silver vs Matte Black)
5. Description	**10%**	Jaccard Word Overlap	स्टिकर्स, कव्हर, ओरखडे यांसारख्या खुणा तपासणे
6. Date Proximity	**10%**	Exponential Day Decay	एकाच दिवशी हरवलेली व सापडलेली वस्तू वर रँक करणे
एकूण (Total)	**100%**	**Max-Heap Priority Queue मध्ये सर्वोच्च गुण असलेली वस्तू वर दाखवली जाते**	

6 कोड ट्रेसेबिलिटी संदर्भ तक्ता (Code Traceability Reference)

DSA CONCEPT	SOURCE FILE	LINE NUMBERS	C++ FUNCTION / STRUCTURE SIGNATURE
Merge Sort	`src/dsa_engine.hpp`	Lines 295 - 350	`void mergeSort(vector<Item>&, string, bool, SortMetrics&)`
Insertion Sort	`src/dsa_engine.hpp`	Lines 246 - 290	`void insertionSort(vector<Item>&, string, bool, SortMetrics&)`
Bubble Sort	`src/dsa_engine.hpp`	Lines 209 - 238	`void bubbleSort(vector<Item>&, string, bool, SortMetrics&)`
Linear Search	`src/dsa_engine.hpp`	Lines 365 - 415	`vector<Item> linearSearch(vector<Item>, q, cat, loc, status, metrics)`
Binary Search	`src/dsa_engine.hpp`	Lines 420 - 480	`vector<Item> binarySearchByName(vector<Item>, q, metrics)`
Priority Queue (Max-Heap)	`src/dsa_engine.hpp`	Lines 510 - 580	`priority_queue<MatchResult, vector, MatchComparator>`
Levenshtein Distance (DP)	`src/dsa_engine.hpp`	Lines 67 - 106	`int levenshteinDistance(const string&, const string&)`
Jaccard Token Similarity	`src/dsa_engine.hpp`	Lines 114 - 145	`double jaccardSimilarity(const string&, const string&)`
Category Hash Index	`src/dsa_engine.hpp`	Lines 150 - 162	`unordered_map<string, vector<int>> buildCategoryIndex()`
Functor Operator Overload	`src/models.hpp`	Lines 75 - 81	`bool MatchComparator::operator()(const MatchResult&, const MatchResult&)`

7 मॅडम विचारू शकतील असे टॉप १० Viva Questions & Answers

Q1. Why did you choose Merge Sort as the primary catalog sorting algorithm?

Answer: Merge Sort guarantees `O(N log N)` time complexity in all cases (Best, Average, and Worst). Furthermore, it is a **stable sorting algorithm**. In a lost and found database, preserving the original registration order for items with identical dates or categories is essential, which Merge Sort achieves reliably.

Q2. What is the fundamental prerequisite of Binary Search, and how is it handled in your application?

Answer: Binary Search strictly requires the underlying dataset to be sorted on the key being searched. In our `/search` route controller, before invoking `dsa::binarySearchByName()`, we automatically sort the array alphabetically using `dsa::mergeSort()` on the `name` field.

Q3. Why is a Priority Queue (Max-Heap) better than QuickSort for Auto-Matching?

Answer: Comparing N lost items with M found items yields `N × M` pairs. Fully sorting all pairs with QuickSort costs `O(K log K)` where `K = N × M`. However, users only need the top 5 highest-probability matches. A Max-Heap extracts the top elements in `O(K log 5)` time and maintains the highest score at the root in `O(1)`, saving memory and CPU cycles.

Q4. How does your system match items with typographical or spelling errors?

Answer: We implemented the **Levenshtein Distance algorithm** using Dynamic Programming in `O(M × N)` time. It calculates the minimum edit operations (insertions, deletions, substitutions) required to transform one string into another, converting it into a normalized ratio from 0.0 to 1.0.

Q5. What happens if words are reordered (e.g. "HP Silver Laptop" vs "Laptop Silver HP")?

Answer: Levenshtein distance penalizes character shifts heavily. To overcome this, we integrated **Jaccard Token Similarity** ($|A \cap B| / |A \cup B|$). By splitting strings into token sets, word order variations achieve a 100% match. Our hybrid matcher combines 60% Jaccard with 40% Levenshtein.

Q6. What is the space complexity trade-off between Merge Sort and Bubble/Insertion Sort?

Answer: Merge Sort requires `O(N)` auxiliary space to hold merged partitions in memory. In contrast, Bubble Sort and Insertion Sort operate in-place with `O(1)` auxiliary memory overhead.

Q7. How does the Bubble Sort early exit optimization work in code?

Answer: We maintain a boolean flag `swapped = false`. If a complete pass through the vector occurs without any adjacent pair swap, the vector is already ordered, and the loop breaks immediately, reducing the Best Case complexity from `O(N²)` to `O(N)`.

Q8. Where and why did you use Hash Tables (`std::unordered_map`)?

Answer: We used Hash Tables in: (1) `dsa::buildCategoryIndex()` to create an inverted index mapping categories to item IDs for `O(1)` subset queries, and (2) `html::tr()` for `O(1)` bilingual Marathi/English translation lookups.

Q9. How do you measure and display microsecond execution times?

Answer: We use C++17 `std::chrono::high_resolution_clock::now()` before and after algorithm execution. We compute duration in microseconds (`µs`) alongside counter variables tracking exact comparisons and swaps, rendering them directly into HTML templates.

Q10. How does the architecture prevent student fraud while preserving DSA transparency?

Answer: We implemented **Zero-Knowledge Blind Custody (Role-Based Access Control)**. Regular students only lodge lost reports and query their private status using unique cryptographic Claim Tokens (`/track`), blocking unauthorized viewing of custody items. Meanwhile, authorized staff (`/admin`) have exclusive access to the Master Catalog and DSA comparison tools (`/search` and `/items`) for institutional oversight.